

Run a standalone Lecture Deconstruction & Cognitive Synthesis on the current lecture material:
\$ARGUMENTS

Input Validation

- `$ARGUMENTS` must be the base filename of the lecture without the extension (e.g., `Lecture_01`, `Lecture_02`).
- If `$ARGUMENTS` is empty, prompt the user to provide a lecture filename.

Execution Instructions

Execute the following pipeline strictly step-by-step to process the entire lecture.

Step 1: Local Context Hydration (History & Current Ingestion)

- **Historical State (N-1 Dependency):** Scan the `output/cs336/` directory for the immediately preceding lecture's artifacts (e.g., if `$ARGUMENTS` is `Lecture_03`, locate `Lecture_01`, `Lecture_02`). Read its `01_Phase1_Architect.md` and `02_Phase2_Educator.md` into context. If none exist (e.g., lecture 1), proceed with an empty state.
- **Current Material:** Read the ENTIRE local lecture file strictly from `E:\allwork\cs336\lecture\$ARGUMENTS.md`.
- **Objective:** Ingest the prior system state and the current document. Do not summarize yet. Hold the raw data in context for downstream Agents.

Step 2: Phase 1 - Agent A (The Architect) Structural & Technical Extraction

Launch the Architect Agent (Backend Logic):

- **System Profile & Rules:** Read and strictly follow the execution framework in `shared/agents/agentA.md`.
- **Writing Style:** Read and strictly apply `shared/writing_style.md`.
- **Inputs:**
 1. The Historical State (from Step 1).
 2. The full text of `E:\allwork\cs336\lecture\$ARGUMENTS.md`.
- **Output:** Write the resulting technical extraction strictly to `output/cs336/$ARGUMENTS/01_Phase1_Architect.md`.

Step 3: Phase 2 - Agent B (The Educator) Cognitive Synthesis

Launch the Educator Agent (Frontend Cognition):

- **System Profile & Rules:** Read and strictly follow the execution framework in `shared/agents/agentB.md`.
- **Writing Style:** Read and strictly apply `shared/writing_style.md`.
- **Inputs:**
 1. The Historical State (from Step 1).
 2. The raw lecture `E:\allwork\cs336\lecture\$ARGUMENTS.md`.
 3. The output from Step 2: `output/cs336/$ARGUMENTS/01_Phase1_Architect.md`.

- **Output:** Write the resulting cognitive and memory models strictly to `output/cs336/$ARGUMENTS/02_Phase2_Educator.md`.

Step 4: Phase 3 - Socratic Loop Initialization

- Print a terminal/chat message:
*"Lecture deconstruction for `$ARGUMENTS` is complete."
 - Read `output/cs336/$ARGUMENTS/01_Phase1_Architect.md` for the structural/technical overview.
 - Read `output/cs336/$ARGUMENTS/02_Phase2_Educator.md` for the cognitive and memory models.System is now in Phase 3. You can now ask any questions about the lecture's logic, or paste your PyTorch code if you are trying to implement it."*